NAMA : Muhammad Rafi' Abdurrahman

NIM : 18224004

**scroll hingga bawah, setiap halaman ada kodingan*

Latihan 1. Skema Pengulangan (1)

1. Mengisikan elemen sebuah array of integer hingga penuh, diketahui indeks awal 1 dan indeks akhir array 100.

   traversal

2. Menuliskan ke layar sebuah kata "Hello" sebanyak n kali, n masukan dari pengguna.

   repeat n

```
PROGRAM IsiArray
{mengisi 100 array hingga penuh}
KAMUS
array[100], temp: integer
ALGORITMA
i transversal [1...100]
     input (temp)
     array[i] ← temp
```

```
PROGRAM HelloN
{Mengoutput hello sebanyak N kali}
KAMUS
N : integer
ALGORITMA
input (N)
repeat N times
     output("Hello")
```

Latihan 1. Skema Pengulangan (2)

iterate-stop

3. Memunculkan sebuah menu ke layar, yaitu:
   - Menampilkan “Hello World” di layar,
   - Membaca 2 integer dan menampilkan hasil penjumlahannya
   - Keluar

   Program akan selesai hanya jika pengguna memilih menu “Keluar”. Jika memilih menu yang lain, program akan mengeksekusi perintah menu tersebut, lalu kembali ke tampilan menu, untuk selanjutnya diulangi lagi proses yang sama.

```
PROGRAM Menu

{membuat menu Hello word, menambah dua integer, dan keluar}

KAMUS

choice, a, b : integer

ALGORITMA

iterate

     output(“1. Katakan Halo! \n 2. Tambah dua angka \n 3.
     Keluar \n Masukkan pilihan anda:\n)

     input (choice)

stop (choice = 3)

if (choice = 1)

     output(“Hello!”)

else if (choice = 2)

     output(“Masukkan dua angka : X Y”)

     input (a,b)

     output(a+b)
```

Latihan 1. Skema Pengulangan (3)

4. Mengisi sebuah array of integer dengan ukuran N. Pengisian array diakhiri jika pengguna memasukkan angka 9999.
   - Array mungkin menjadi kosong (jika angka yang dimasukkan pertama kali 9999).
   - Tidak ada penanganan khusus untuk kasus array kosong

while-do

```
PROGRAM IsiNArray

{mengisi array integer dengan kapasitas N dan berhenti ketika
input = 9999}

KAMUS

array[1...N], N, temp, i : integer

ALGORITMA

input(N)

i ← 1

iterate

     input (temp)

stop (temp = 9999)

     array[i] ← temp

     i ← i+1
```

- Buatlah program dalam notasi algoritmik, untuk membaca sejumlah nilai UTS mahasiswa di suatu kelas. Nilai UTS yang valid adalah 0..100. Pembacaan dihentikan jika **masukan nilai UTS di luar range nilai yang diizinkan**
- Di akhir program dihitung dan ditampilkan rata-rata nilai UTS seluruh mahasiswa di kelas.
- Jika tidak ada nilai UTS yang dimasukkan, tuliskan “Tidak ada data”

```
PROGRAM InputDataUTS

{membaca nilai UTS, bila tidak valid !(0-100), stop membaca dan
mengeluarkan nilai rata-rata}

KAMUS

temp, sum, count : integer

ALGORITMA

iterate

     output(“Nilai UTS = “)

     input(temp)

stop (not(0<=temp<=100)

     sum ← sum + temp

     count ← count+1

if (count>0)

     output(“Nilai rata-rata UTS = “, sum/count)

else

     output("Tidak ada data”)
```

- Buatlah program dalam notasi algoritmik yang menerima 3 buah bilangan integer yaitu h, m, dan s yang akan digunakan untuk membentuk data bertype jam. Definisi type jam adalah sbb.

```
type jam : < HH : integer[0..23];  { bagian jam }
             MM : integer[0..59];  { bagian menit }
             SS : integer[0..59] > { bagian detik }
```

- **Jika ketiga input tidak valid**, dituliskan pesan kesalahan ke layar "Tidak dapat membentuk jam" dan pemasukan data h, m, s diulangi sampai didapatkan nilai yang valid.
- **Jika ketiga input valid**, maka sebuah variabel J bertype jam akan terbentu (didefinisikan nilainya) dengan J.HH bernilai h, J.MM bernilai m, J.SS bernilai s.
- Nilai valid didefinisikan sebagai: $0 \le h \le 23$; $0 \le m \le 59$; $0 \le s \le 59$

```
PROGRAM Jam
{membuat data bertipe jam dengan input integer h,m, dan s}
KAMUS
type jam : <   HH : integer[0..23];
               MM : integer[0..59];
               SS : integer[0..59];
h,m,s : integer
jamUser : jam
ALGORITMA
iterate
     input(h)
     input(m)
     input(s)
stop(0<=h<=23 and 0<=m<=59 and 0<=s<=59)
     output("Tidak dapat membentuk jam, ulangi input!")


jamUser.HH ← h
jamUser.MM ← m
jamUser.SS ← s
```

Latihan 4 (1)

- Buatlah program dalam notasi algoritmik, untuk membaca nilai UTS dan nilai UAS mahasiswa untuk setiap pelajaran yang diikutinya (0..100) **dan diakhiri jika nilai masukan UTS di luar range nilai yang diizinkan**, kemudian menghitung dan mencetak rata-rata nilai akhir dari seluruh pelajaran.
- Gunakan skema validasi data untuk memastikan nilai UAS pada range 0..100 (jika data tidak memenuhi syarat, read data UAS diulang). Nilai akhir untuk suatu pelajaran dihitung dari rumus (40% * nilai UTS) + (60% * nilai UAS).

Codingan ada di page selanjutnya!

```
PROGRAM NilaiUTSUAS

{membaca nilai UTS UAS, bila input UTS diluar range, buat rata-
rata semua pelajaran, bila tidak, lanjutkan ke nilai UAS. Nilai
UAS diminta input hingga input valid, setelah itu buat nilai
akhir per-perlajaran setiap selesai menginput nilai UAS.Nilai
pelajaran = (40% * nilai UTS) + (60% * nilai UAS)}

KAMUS

UTS, UAS, tempPel, sum, count : integer

ALGORITMA

count ← 0

iterate

     input(UTS)

stop (UTS < 0 or UTS > 100)

     iterate

          input(UAS)

     stop (0<=UAS<=100)

          output("Ulangi input nilai (0..100)!")

     tempPel ← (UTS * 4/10) + (UAS * 6/10)

     count ← count + 1

     output("Nilai akhir pelajaran ", count,"= " tempPel)

     sum ← sum + tempPel

if (count>0)

     output("Nilai rata-rata dari ", count," pelajaran adalah =
     ", sum/count)

else

     output("Data kosong, tidak ada nilai rata-rata!")
```